

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені Ігоря Сікорського»**

**Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій**

**Звіт до комп'ютерного практикуму №2
З дисципліни «Комп'ютерне моделювання дискретно-подійних систем»
«Об'єктно-орієнтований підхід до побудови імітаційних моделей
дискретно-подійних систем»**

Прийняв:

Викладач

Коринецький Дмитро Сергійович

«05» жовтня 2026 р.

Виконав:

Студент 4 курсу, гр. ПІ-34

Скачков Іван

Олексійович

2026 р.

Зміст

Лабораторна робота №2.....	3
Технічне завдання	4
Виконання	5
Структура програми.....	5
Лістинг файлів	6
Завдання 1–2	11
Лістинг файлів	11
Результат роботи	12
Висновок	13
Завдання 3–4	14
Лістинг файлів	14
Результат роботи	16
Висновок	18
Завдання 5	19
Лістинг файлів	19
Результат роботи	20
Висновок	21
Завдання 6	22
Лістинг файлів	23
Результат роботи	24
Висновок	25
Висновок	26
Посилання	27

Лабораторна робота №2

Тема: Об'єктно-орієнтований підхід до побудови імітаційних моделей дискретно-подійних систем.

Мета: здобути практичні навички побудови алгоритму імітації мереж масового обслуговування з використанням об'єктно-орієнтованого підходу, збирання статистики про роботу моделі та верифікації моделі.

Мова реалізації: Python 3.12, лише стандартна бібліотека. Структура класів відповідає наведеній у методичних вказівках (Element, Create, Process, Model, FunRand) і розширена відповідно до завдань.

Технічне завдання

1. Реалізувати алгоритм імітації простої моделі обслуговування одним пристроєм з використанням об'єктно-орієнтованого підходу.
2. Модифікувати алгоритм, додавши обчислення середнього завантаження пристрою.
3. Створити модель за схемою, представленою на рисунку 1.
4. Виконати верифікацію моделі, змінюючи значення вхідних змінних та параметрів моделі. Навести результати верифікації у таблиці.
5. Модифікувати клас PROCESS, щоб можна було його використовувати для моделювання процесу обслуговування кількома ідентичними пристроями.
6. Модифікувати клас PROCESS, щоб можна було організовувати вихід в два і більше наступних блоків, в тому числі з поверненням у попередні блоки.



Рисунок 1 – Схема моделі

Виконання

Структура програми

Алгоритм імітації побудовано за принципом найближчої події: модельний час просувається від моменту однієї події до моменту наступної, і після кожного просування виконуються зміни стану моделі, що відповідають цій події. Просування стану моделі орієнтоване на події: подія «надходження вимоги» та подія «закінчення обслуговування в пристрої».

Модель складається з елементів — нащадків універсального класу `Element`. Кожний елемент зберігає момент своєї найближчої події `tnext` і реалізує дві дії: `in_act()` — вхід вимоги в елемент та `out_act()` — вихід вимоги з елемента. Клас `Model` у циклі знаходить найменше значення `tnext` серед усіх елементів, передає елементам тривалість інтервалу для накопичення статистики, просуває поточний час і викликає `out_act()` для елементів, у яких настала подія. Діаграма класів наведена на рисунку 2, призначення класів — у таблиці 1.

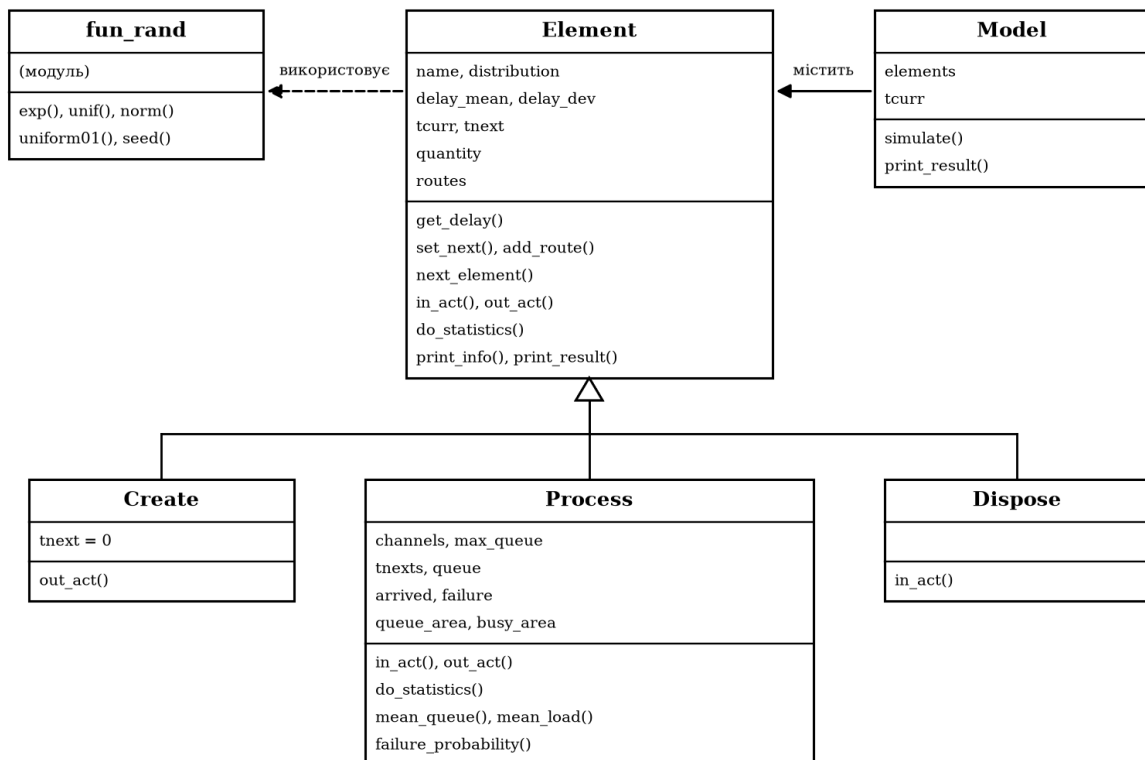


Рисунок 2 – Діаграма класів

Таблиця 1 – Класи та модулі програми

Клас (файл)	Призначення
Element (element.py)	Базовий елемент: параметри часової затримки, лічильник вимог, список маршрутів (елемент, ймовірність), вибір наступного елемента
Create (create.py)	Джерело вимог: у момент події генерує наступний момент надходження і передає вимогу далі
Process (process.py)	Обслуговування: кілька ідентичних пристроїв, обмежена черга, відмови, статистика черги та завантаження
Dispose (dispose.py)	Вихід із моделі: підраховує вимоги, що залишили систему
Model (model.py)	Алгоритм імітації: просування часу за принципом найближчої події, збирання статистики, виведення протоколу та результатів
fun_rand (fun_rand.py)	Генератори випадкових величин: експоненційний, рівномірний, нормальний закони
theory (theory.py)	Аналітичні формули СМО М/М/с/К для верифікації результатів імітації

Порівняно з кодом із методичних вказівок внесено такі зміни. По-перше, у методі `Process.out_act()` вимога після обслуговування передається наступному елементу — без цього неможливо з'єднати кілька процесів у ланцюг. По-друге, додано елемент `Dispose`, який фіксує вихід вимог і дає змогу перевіряти баланс. По-третє, ймовірність відмови обчислюється як відношення кількості відмов до кількості вимог, що надійшли в елемент, а статистика збирається рівно на інтервалі моделювання $[0; T]$.

Лістинг файлів

Файл `fun_rand.py`:

```

"""Генератори випадкових величин (аналог класу FunRand)."""
import math
import random

_rng = random.Random()

def seed(value):
    _rng.seed(value)

def uniform01():
    return _rng.random()

def exp(mean):
    return -mean * math.log(1 - _rng.random())

```

```
def unif(low, high):
    return low + _rng.random() * (high - low)
```

```
def norm(mean, dev):
    return mean + dev * _rng.gauss(0, 1)
```

Файл element.py:

```
"""Базовий елемент моделі: часова затримка, лічильник і маршрути до наступних елементів."""
import math
```

```
import fun_rand
```

```
class Element:
    def __init__(self, name, delay_mean=1.0, delay_dev=0.0, distribution="exp"):
        self.name = name
        self.delay_mean = delay_mean
        self.delay_dev = delay_dev
        self.distribution = distribution
        self.tcurr = 0.0
        self.tnext = math.inf
        self.quantity = 0
        self.routes = []

    def get_delay(self):
        if self.distribution == "exp":
            return fun_rand.exp(self.delay_mean)
        if self.distribution == "norm":
            return fun_rand.norm(self.delay_mean, self.delay_dev)
        if self.distribution == "unif":
            return fun_rand.unif(self.delay_mean, self.delay_dev)
        return self.delay_mean

    def set_next(self, element):
        self.routes = [(element, 1.0)]

    def add_route(self, element, probability):
        self.routes.append((element, probability))

    def next_element(self):
        """Вибір наступного елемента за ймовірностями маршрутів."""
        r, cumulative = fun_rand.uniform01(), 0.0
        for element, probability in self.routes:
            cumulative += probability
            if r < cumulative:
                return element
        return None

    def in_act(self):
        pass

    def out_act(self):
        self.quantity += 1

    def do_statistics(self, delta):
        pass
```

```

def print_info(self):
    print(f" {self.name}: quantity = {self.quantity}, tnext = {self.tnext:.4f}")

def print_result(self, time):
    print(f"{self.name}: quantity = {self.quantity}")

```

Файл create.py:

```

"""CREATE: джерело вимог."""
from element import Element

class Create(Element):
    def __init__(self, name, delay_mean, delay_dev=0.0, distribution="exp"):
        super().__init__(name, delay_mean, delay_dev, distribution)
        self.tnext = 0.0 # імітація починається з надходження вимоги

    def out_act(self):
        super().out_act()
        self.tnext = self.tcurr + self.get_delay()
        target = self.next_element()
        if target:
            target.in_act()

```

Файл process.py:

```

"""PROCESS: обслуговування кількома ідентичними пристроями з обмеженою чергою."""
import math

from element import Element

class Process(Element):
    def __init__(self, name, delay_mean, channels=1, max_queue=math.inf,
                 delay_dev=0.0, distribution="exp"):
        super().__init__(name, delay_mean, delay_dev, distribution)
        self.channels = channels
        self.max_queue = max_queue
        self.tnexts = [math.inf] * channels # момент звільнення кожного пристрою
        self.queue = 0
        self.arrived = 0
        self.failure = 0
        self.queue_area = 0.0
        self.busy_area = 0.0

    @property
    def busy(self):
        return sum(t != math.inf for t in self.tnexts)

    def in_act(self):
        self.arrived += 1
        if self.busy < self.channels:
            self.tnexts[self.tnexts.index(math.inf)] = self.tcurr + self.get_delay()
            self.tnext = min(self.tnexts)
        elif self.queue < self.max_queue:
            self.queue += 1
        else:
            self.failure += 1

    def out_act(self):
        for i, t in enumerate(self.tnexts):

```

```

        if t != self.tcurr:
            continue
        super().out_act()
        self.tnexts[i] = math.inf
        if self.queue > 0:
            self.queue -= 1
            self.tnexts[i] = self.tcurr + self.get_delay()
        self.tnext = min(self.tnexts)
        target = self.next_element()
        if target:
            target.in_act()

def do_statistics(self, delta):
    self.queue_area += self.queue * delta
    self.busy_area += self.busy * delta

def mean_queue(self, time):
    return self.queue_area / time

def mean_load(self, time):
    """Середня частка зайнятих пристроїв."""
    return self.busy_area / time / self.channels

def failure_probability(self):
    return self.failure / self.arrived if self.arrived else 0.0

def print_info(self):
    print(f" {self.name}: busy = {self.busy}/{self.channels}, queue = {self.queue}, "
          f"quantity = {self.quantity}, failure = {self.failure}, tnext = "
          f"{self.tnext:.4f}")

def print_result(self, time):
    print(f"{self.name}: arrived = {self.arrived}, quantity = {self.quantity}, "
          f"failure = {self.failure}")
    print(f"    mean queue = {self.mean_queue(time):.4f}, mean load = "
          f"{self.mean_load(time):.4f}, "
          f"failure probability = {self.failure_probability():.4f}")

```

Файл dispose.py:

```

"""DESPOSE: вихід вимог із моделі."""
from element import Element

```

```

class Dispose(Element):
    def in_act(self):
        self.quantity += 1

```

Файл model.py:

```

"""Model: просування модельного часу за принципом найближчої події."""

```

```

class Model:
    def __init__(self, elements):
        self.elements = elements
        self.tcurr = 0.0

    def simulate(self, time, verbose_events=0):
        """verbose_events – скільки перших подій вивести у протокол."""
        step = 0

```

```

while True:
    tnext = min(e.tnext for e in self.elements)
    if tnext > time: # статистика збирається рівно на інтервалі [0; time]
        tnext = time
    for e in self.elements:
        e.do_statistics(tnext - self.tcurr)
    self.tcurr = tnext
    if self.tcurr >= time:
        break
    for e in self.elements:
        e.tcurr = self.tcurr
    for e in self.elements:
        if e.tnext == self.tcurr:
            if step < verbose_events:
                print(f"\nПодія в {e.name}, t = {self.tcurr:.4f}")
            e.out_act()
    if step < verbose_events:
        for e in self.elements:
            e.print_info()
    step += 1

def print_result(self):
    print(f"\n----- РЕЗУЛЬТАТИ (T = {self.tcurr:g}) -----")
    for e in self.elements:
        e.print_result(self.tcurr)

```

Файл theory.py:

```

"""Аналітичні характеристики СМО М/М/с/К для перевірки результатів імітації."""

def mmck(arrival_mean, service_mean, channels=1, max_queue=1000):
    """Повертає (ймовірність відмови, середня черга, середнє завантаження пристрою).

    arrival_mean – середній інтервал між вимогами, service_mean – середній час
    обслуговування.
    Нескінченна черга наближається великим max_queue (при завантаженні < 1).
    """
    a = service_mean / arrival_mean
    weights = [1.0]
    for n in range(1, channels + max_queue + 1):
        weights.append(weights[-1] * a / min(n, channels))
    total = sum(weights)
    p = [w / total for w in weights]
    p_fail = p[-1]
    mean_queue = sum((n - channels) * p[n] for n in range(channels + 1, len(p)))
    mean_load = sum(min(n, channels) * p[n] for n in range(len(p))) / channels
    return p_fail, mean_queue, mean_load

```

Завдання 1–2

Модель обслуговування одним пристроєм (рисунок 3): елемент CREATE генерує вимоги з експоненційно розподіленим інтервалом (середнє 2,0), елемент PROCESS обслуговує їх одним пристроєм з експоненційним часом обслуговування (середнє 1,0), черга обмежена п'ятьма місцями. Якщо пристрій зайнятий і черга заповнена, вимога отримує відмову.



Рисунок 3 – Структура моделі з одним пристроєм

Для завдання 2 у метод `Process.do_statistics()` додано накопичення величини $\text{busy} \cdot \Delta t$, де busy — кількість зайнятих пристроїв на інтервалі між подіями довжиною Δt . Середнє завантаження пристрою обчислюється як $R = \Sigma(\text{busy} \cdot \Delta t) / (T \cdot c)$, де c — кількість пристроїв. Аналогічно середня довжина черги $L = \Sigma(\text{queue} \cdot \Delta t) / T$.

Лістинг файлів

Файл `task1_single.py`:

```
"""Завдання 1-2: модель обслуговування одним пристроєм, середнє завантаження пристрою."""
import fun_rand
from create import Create
from dispose import Dispose
from model import Model
from process import Process
from theory import mmck

DELAY_CREATE, DELAY_PROCESS, MAX_QUEUE, TIME = 2.0, 1.0, 5, 100_000

def run(verbose_events=0):
    fun_rand.seed(1)
    create = Create("CREATE", DELAY_CREATE)
    process = Process("PROCESS", DELAY_PROCESS, max_queue=MAX_QUEUE)
    dispose = Dispose("DESPOSE")
    create.set_next(process)
    process.set_next(dispose)
    model = Model([create, process, dispose])
    model.simulate(TIME, verbose_events)
    return model, create, process, dispose
```

```

def main():
    model, _, process, _ = run(verbose_events=4)
    model.print_result()

    p_fail, mean_queue, mean_load = mmck(Delay_create, Delay_process, 1, MAX_QUEUE)
    print("\nПорівняння з аналітичною моделлю М/М/1/К:")
    print(f"{'':<24}{'імітація':>10}{'теорія':>10}")
    print(f"{'середнє завантаження':<24}{process.mean_load(TIME):>10.4f}{mean_load:>10.4f}")
    print(f"{'середня довжина черги':<24}{process.mean_queue(TIME):>10.4f}{mean_queue:>10.4f}")
    print(f"{'ймовірність відмови':<24}{process.failure_probability():>10.4f}{p_fail:>10.4f}")

if __name__ == "__main__":
    main()

```

Результат роботи

```

Подія в CREATE, t = 0.0000
CREATE: quantity = 1, tnext = 0.2886
PROCESS: busy = 1/1, queue = 0, quantity = 0, failure = 0, tnext = 1.4430
DESPOSE: quantity = 0, tnext = inf

Подія в CREATE, t = 0.2886
CREATE: quantity = 2, tnext = 0.8775
PROCESS: busy = 1/1, queue = 1, quantity = 0, failure = 0, tnext = 1.4430
DESPOSE: quantity = 0, tnext = inf

Подія в CREATE, t = 0.8775
CREATE: quantity = 3, tnext = 2.0713
PROCESS: busy = 1/1, queue = 2, quantity = 0, failure = 0, tnext = 1.4430
DESPOSE: quantity = 0, tnext = inf

Подія в PROCESS, t = 1.4430
CREATE: quantity = 3, tnext = 2.0713
PROCESS: busy = 1/1, queue = 1, quantity = 1, failure = 0, tnext = 2.9976
DESPOSE: quantity = 1, tnext = inf

----- РЕЗУЛЬТАТИ (T = 100000) -----
CREATE: quantity = 49930
PROCESS: arrived = 49930, quantity = 49554, failure = 376
        mean queue = 0.4519, mean load = 0.4965, failure probability = 0.0075
DESPOSE: quantity = 49554

Порівняння з аналітичною моделлю М/М/1/К:
        імітація    теорія
середнє завантаження      0.4965  0.4961
середня довжина черги     0.4519  0.4488
ймовірність відмови       0.0075  0.0079

```

Таблиця 2 – Порівняння результатів імітації з аналітичною моделлю М/М/1/К (T = 100000)

Показник	Імітація	Теорія
Середнє завантаження пристрою	0,4965	0,4961
Середня довжина черги	0,4519	0,4488
Ймовірність відмови	0,0075	0,0079

Висновок

З протоколу перших подій видно, що алгоритм працює коректно: вимога, яка надійшла у вільний пристрій, одразу займає його, наступні стають у чергу, а після закінчення обслуговування перша вимога з черги переходить у пристрій. За час моделювання $T = 100000$ створено 49930 вимог, з них обслужено 49554, відмову отримали 376, у системі залишилось 0. Баланс вимог виконується: $49930 = 49554 + 376 + 0$.

Середнє завантаження пристрою дорівнює 0,4965, тобто пристрій зайнятий приблизно половину часу, що відповідає співвідношенню середнього часу обслуговування і середнього інтервалу надходження $1,0 / 2,0$. Результати імітації збігаються з аналітичними значеннями для системи М/М/1/К (таблиця 2), що підтверджує правильність алгоритму і розрахунку статистики.

Завдання 3–4

Модель за схемою з рисунка 1 складається з джерела вимог і трьох послідовних одноканальних процесів з обмеженими чергами. Вимога, що отримала відмову в будь-якому процесі, залишає модель. У базовому варіанті середній інтервал надходження дорівнює 2,0, середній час обслуговування в кожному процесі 1,0, максимальна довжина кожної черги 5; усі затримки розподілені експоненційно.

Верифікація виконувалась трьома способами. Перший — перевірка закону збереження вимог: кількість вимог, що надійшли в елемент, має дорівнювати сумі обслужених, відмовлених і тих, що залишились у черзі та в пристрої, а кількість обслужених попереднім елементом — кількості тих, що надійшли в наступний. Другий — зміна по одному вхідному параметру і перевірка, чи змінюються вихідні характеристики в очікуваному напрямку. Третій — порівняння характеристик першого процесу з точними аналітичними формулами системи M/M/1/K (на вхід PROCESS 1 надходить найпростіший потік, тому теорія для нього точна).

Лістинг файлів

Файл task3_chain.py:

```
"""Завдання 3-4: модель CREATE -> PROCESS 1 -> PROCESS 2 -> PROCESS 3 -> DESPOSE та її
верифікація."""
import fun_rand
from create import Create
from dispose import Dispose
from model import Model
from process import Process
from theory import mmck

TIME = 100_000

# № прогону: (delay CREATE, delay PROCESS 1..3, max_queue)
RUNS = [
    (2.0, 1.0, 1.0, 1.0, 5),
    (1.0, 1.0, 1.0, 1.0, 5),
    (4.0, 1.0, 1.0, 1.0, 5),
    (2.0, 2.0, 1.0, 1.0, 5),
    (2.0, 1.0, 2.0, 1.0, 5),
    (2.0, 1.0, 1.0, 2.0, 5),
    (2.0, 1.0, 1.0, 1.0, 0),
    (2.0, 1.0, 1.0, 1.0, 20),
    (2.0, 0.5, 0.5, 0.5, 5),
]
```

```

def run(delay_create, d1, d2, d3, max_queue, time=TIME, verbose_events=0):
    fun_rand.seed(1)
    create = Create("CREATE", delay_create)
    processes = [Process(f"PROCESS {i}", d, max_queue=max_queue) for i, d in
enumerate((d1, d2, d3), 1)]
    dispose = Dispose("DESPOSE")
    chain = [create, *processes, dispose]
    for element, following in zip(chain, chain[1:]):
        element.set_next(following)
    model = Model(chain)
    model.simulate(time, verbose_events)
    return model, create, processes, dispose

def balance_ok(create, processes, dispose):
    """Закон збереження вимог: кожна вимога або обслужена, або відмовлена, або ще в
елементі."""
    inputs = [create.quantity] + [p.quantity for p in processes[:-1]]
    return (all(p.arrived == n for p, n in zip(processes, inputs))
        and all(p.arrived == p.quantity + p.failure + p.queue + p.busy for p in
processes)
        and dispose.quantity == processes[-1].quantity)

def main():
    model, *_ = run(*RUNS[0], verbose_events=5)
    model.print_result()

    print(f"\nВерифікація моделі (T = {TIME}). Таблиця А – вхідні параметри, кількості,
відмови:")
    print(f"{'№':>2} {'tC':>4} {'t1':>4} {'t2':>4} {'t3':>4} {'maxQ':>4} |
{'створено':>8} {'вийшло':>7} | "
        f"{'Рвідм1':>7} {'Рвідм2':>7} {'Рвідм3':>7} | баланс")
    results = []
    for number, params in enumerate(RUNS, 1):
        model, create, processes, dispose = run(*params)
        results.append((number, params, processes))
        fails = " ".join(f"{p.failure_probability():>7.4f}" for p in processes)
        print(f"{number:>2} {params[0]:>4} {params[1]:>4} {params[2]:>4} {params[3]:>4}
{params[4]:>4} | "
            f"{create.quantity:>8} {dispose.quantity:>7} | {fails} | "
            f"{'так' if balance_ok(create, processes, dispose) else 'НІ'}")

    print("\nТаблиця В – середнє завантаження пристроїв і середні довжини черг:")
    print(f"{'№':>2} | {'R1':>6} {'R2':>6} {'R3':>6} | {'L1':>7} {'L2':>7} {'L3':>7}")
    for number, _, processes in results:
        loads = " ".join(f"{p.mean_load(TIME):>6.3f}" for p in processes)
        queues = " ".join(f"{p.mean_queue(TIME):>7.3f}" for p in processes)
        print(f"{number:>2} | {loads} | {queues}")

    print("\nТаблиця В – PROCESS 1: імітація / теорія М/М/1/К:")
    print(f"{'№':>2} | {'Рвідм':>15} | {'завантаження':>15} | {'черга':>15}")
    for number, params, processes in results:
        p = processes[0]
        p_fail, mean_queue, mean_load = mmck(params[0], params[1], 1, params[4])
        print(f"{number:>2} | {p.failure_probability():>7.4f}/{p_fail:<7.4f} | "
            f"{p.mean_load(TIME):>7.4f}/{mean_load:<7.4f} | "
            f"{p.mean_queue(TIME):>7.4f}/{mean_queue:<7.4f}")

if __name__ == "__main__":
    main()

```

Результат роботи

Подія в CREATE, t = 0.0000

CREATE: quantity = 1, tnext = 0.2886
PROCESS 1: busy = 1/1, queue = 0, quantity = 0, failure = 0, tnext = 1.4430
PROCESS 2: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
PROCESS 3: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
DESPOSE: quantity = 0, tnext = inf

Подія в CREATE, t = 0.2886

CREATE: quantity = 2, tnext = 0.8775
PROCESS 1: busy = 1/1, queue = 1, quantity = 0, failure = 0, tnext = 1.4430
PROCESS 2: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
PROCESS 3: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
DESPOSE: quantity = 0, tnext = inf

Подія в CREATE, t = 0.8775

CREATE: quantity = 3, tnext = 2.0713
PROCESS 1: busy = 1/1, queue = 2, quantity = 0, failure = 0, tnext = 1.4430
PROCESS 2: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
PROCESS 3: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
DESPOSE: quantity = 0, tnext = inf

Подія в PROCESS 1, t = 1.4430

CREATE: quantity = 3, tnext = 2.0713
PROCESS 1: busy = 1/1, queue = 1, quantity = 1, failure = 0, tnext = 2.9976
PROCESS 2: busy = 1/1, queue = 0, quantity = 0, failure = 0, tnext = 1.4717
PROCESS 3: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
DESPOSE: quantity = 0, tnext = inf

Подія в PROCESS 2, t = 1.4717

CREATE: quantity = 3, tnext = 2.0713
PROCESS 1: busy = 1/1, queue = 1, quantity = 1, failure = 0, tnext = 2.9976
PROCESS 2: busy = 0/1, queue = 0, quantity = 1, failure = 0, tnext = inf
PROCESS 3: busy = 1/1, queue = 0, quantity = 0, failure = 0, tnext = 2.0387
DESPOSE: quantity = 0, tnext = inf

----- РЕЗУЛЬТАТИ (T = 100000) -----

CREATE: quantity = 49807
PROCESS 1: arrived = 49807, quantity = 49383, failure = 423
mean queue = 0.4495, mean load = 0.4962, failure probability = 0.0085
PROCESS 2: arrived = 49383, quantity = 49047, failure = 335
mean queue = 0.4390, mean load = 0.4929, failure probability = 0.0068
PROCESS 3: arrived = 49047, quantity = 48762, failure = 285
mean queue = 0.4229, mean load = 0.4881, failure probability = 0.0058
DESPOSE: quantity = 48762

Верифікація моделі (T = 100000). Таблиця А – вхідні параметри, кількості, відмови:

№	tC	t1	t2	t3	maxQ	створено	вийшло	Рвідм1	Рвідм2	Рвідм3	баланс
1	2.0	1.0	1.0	1.0	5	49807	48762	0.0085	0.0068	0.0058	так
2	1.0	1.0	1.0	1.0	5	99984	75173	0.1435	0.0738	0.0522	так
3	4.0	1.0	1.0	1.0	5	25313	25304	0.0001	0.0001	0.0001	так
4	2.0	2.0	1.0	1.0	5	50046	42642	0.1446	0.0018	0.0021	так
5	2.0	1.0	2.0	1.0	5	50116	42524	0.0076	0.1432	0.0019	так
6	2.0	1.0	1.0	2.0	5	50032	42720	0.0084	0.0066	0.1331	так
7	2.0	1.0	1.0	1.0	0	49767	24403	0.3340	0.1677	0.1154	так
8	2.0	1.0	1.0	1.0	20	49988	49985	0.0000	0.0000	0.0000	так
9	2.0	0.5	0.5	0.5	5	50231	50209	0.0001	0.0001	0.0002	так

Таблиця Б – середнє завантаження пристроїв і середні довжини черг:

№	R1	R2	R3	L1	L2	L3
1	0.496	0.493	0.488	0.449	0.439	0.423
2	0.859	0.793	0.754	2.143	1.616	1.374
3	0.255	0.252	0.255	0.087	0.084	0.086
4	0.857	0.427	0.426	2.143	0.285	0.285
5	0.499	0.857	0.425	0.461	2.158	0.284
6	0.494	0.496	0.859	0.448	0.448	2.157
7	0.333	0.279	0.243	0.000	0.000	0.000
8	0.498	0.503	0.503	0.489	0.517	0.507
9	0.252	0.251	0.252	0.084	0.084	0.084

Таблиця В – PROCESS 1: імітація / теорія М/М/1/К:

№	Рвідм	завантаження	черга
1	0.0085/0.0079	0.4962/0.4961	0.4495/0.4488
2	0.1435/0.1429	0.8588/0.8571	2.1428/2.1429
3	0.0001/0.0002	0.2554/0.2500	0.0870/0.0830
4	0.1446/0.1429	0.8571/0.8571	2.1431/2.1429
5	0.0076/0.0079	0.4991/0.4961	0.4609/0.4488
6	0.0084/0.0079	0.4938/0.4961	0.4478/0.4488
7	0.3340/0.3333	0.3333/0.3333	0.0000/0.0000
8	0.0000/0.0000	0.4981/0.5000	0.4891/0.5000
9	0.0001/0.0002	0.2521/0.2500	0.0841/0.0830

Таблиця 3 – Верифікація моделі: вхідні параметри, кількості вимог і ймовірності відмови (Т = 100000)

№	tC	t1	t2	t3	maxQ	Створено	Вийшло	Рвідм1	Рвідм2	Рвідм3	Баланс
1	2,0	1,0	1,0	1,0	5	49807	48762	0,0085	0,0068	0,0058	так
2	1,0	1,0	1,0	1,0	5	99984	75173	0,1435	0,0738	0,0522	так
3	4,0	1,0	1,0	1,0	5	25313	25304	0,0001	0,0001	0,0001	так
4	2,0	2,0	1,0	1,0	5	50046	42642	0,1446	0,0018	0,0021	так
5	2,0	1,0	2,0	1,0	5	50116	42524	0,0076	0,1432	0,0019	так
6	2,0	1,0	1,0	2,0	5	50032	42720	0,0084	0,0066	0,1331	так
7	2,0	1,0	1,0	1,0	0	49767	24403	0,3340	0,1677	0,1154	так
8	2,0	1,0	1,0	1,0	20	49988	49985	0,0000	0,0000	0,0000	так
9	2,0	0,5	0,5	0,5	5	50231	50209	0,0001	0,0001	0,0002	так

Позначення: tC — середній інтервал надходження вимог; t1, t2, t3 — середній час обслуговування в процесах; maxQ — максимальна довжина черги; Рвідм — ймовірність відмови.

Таблиця 4 – Верифікація моделі: середнє завантаження пристроїв R і середні довжини черг L

№	R1	R2	R3	L1	L2	L3	Зміна параметра та спостережувана поведінка
1	0,496	0,493	0,488	0,449	0,439	0,423	Базовий варіант
2	0,859	0,793	0,754	2,143	1,616	1,374	Вимоги надходять удвічі частіше: завантаження, черги і відмови зростають у всіх процесах
3	0,255	0,252	0,255	0,087	0,084	0,086	Вимоги надходять удвічі рідше: завантаження $\approx$ 0,25, відмови практично зникають
4	0,857	0,427	0,426	2,143	0,285	0,285	PROCESS 1 удвічі повільніший: він стає «вузьким місцем», наступні процеси розвантажуються
5	0,499	0,857	0,425	0,461	2,158	0,284	«Вузьке місце» — PROCESS 2: відмови зосереджені в ньому
6	0,494	0,496	0,859	0,448	0,448	2,157	«Вузьке місце» — PROCESS 3
7	0,333	0,279	0,243	0,000	0,000	0,000	Черги заборонені: довжина черг 0, відмови максимальні
8	0,498	0,503	0,503	0,489	0,517	0,507	Черга до 20 місць: відмов немає, черги довші
9	0,252	0,251	0,252	0,084	0,084	0,084	Обслуговування вдвічі швидше: завантаження $\approx$ 0,25

Таблиця 5 – PROCESS 1: порівняння імітації з теорією M/M/1/K

№	Рвідм, імітація	Рвідм, теорія	R, імітація	R, теорія	L, імітація	L, теорія
1	0,0085	0,0079	0,4962	0,4961	0,4495	0,4488
2	0,1435	0,1429	0,8588	0,8571	2,1428	2,1429
3	0,0001	0,0002	0,2554	0,2500	0,0870	0,0830
4	0,1446	0,1429	0,8571	0,8571	2,1431	2,1429
5	0,0076	0,0079	0,4991	0,4961	0,4609	0,4488
6	0,0084	0,0079	0,4938	0,4961	0,4478	0,4488
7	0,3340	0,3333	0,3333	0,3333	0,0000	0,0000
8	0,0000	0,0000	0,4981	0,5000	0,4891	0,5000
9	0,0001	0,0002	0,2521	0,2500	0,0841	0,0830

Висновок

Модель із трьох послідовних процесів побудована з тих самих класів, що й модель з одним пристроєм: достатньо створити елементи і з'єднати їх методом `set_next()`. У базовому варіанті створено 49807 вимог, через усі три процеси пройшло 48762; завантаження пристроїв становить 0,496, 0,493 і 0,488. Завантаження дещо знижується від першого процесу до третього, оскільки частина вимог втрачається через відмови й інтенсивність потоку вздовж ланцюга зменшується.

Верифікація підтвердила правильність моделі. У всіх дев'яти прогонах виконується закон збереження вимог. Зміна входних параметрів спричиняє логічно очікувані зміни результатів: збільшення інтенсивності надходження підвищує завантаження, довжину черг і частку відмов; сповільнення одного з процесів робить його «вузьким місцем» і розвантажує наступні; заборона черг зводить їх довжину до нуля і максимально збільшує відмови; збільшення місткості черг усуває відмови ціною довших черг.

Для першого процесу результати імітації збігаються з точними аналітичними значеннями: наприклад, у прогоні 2 ймовірність відмови 0,1435 проти теоретичної 0,1429, середня довжина черги 2,1428 проти 2,1429. Розбіжності не перевищують статистичної похибки імітації.

Завдання 5

Щоб клас `Process` моделював обслуговування кількома ідентичними пристроями, у нього додано параметр `channels` — кількість пристроїв, а замість одного моменту звільнення зберігається список `tnexts` із моментами звільнення кожного пристрою (нескінченність означає, що пристрій вільний). Кількість зайнятих пристроїв `busy` обчислюється за цим списком, а момент найближчої події елемента `tnext` дорівнює найменшому з елементів списку.

Вимога, що надійшла, займає перший вільний пристрій; якщо всі пристрої зайняті, вона стає в чергу або отримує відмову. У методі `out_act()` звільняються ті пристрої, у яких момент завершення збігається з поточним часом, і кожний звільнений пристрій одразу бере наступну вимогу з черги. При `channels = 1` клас працює так само, як одноканальний, тому моделі з попередніх завдань не змінювались.

Для перевірки взято систему з перевантаженим одним пристроєм: середній інтервал надходження 1,0, середній час обслуговування 2,0, черга до 5 місць. Кількість пристроїв змінювалась від 1 до 5, результати порівнювались з аналітичними формулами системи М/М/с/К.

Лістинг файлів

Файл `task5_multichannel.py`:

```
"""Завдання 5: PROCESS з кількома ідентичними пристроями."""
import fun_rand
from create import Create
from dispose import Dispose
from model import Model
from process import Process
from theory import mmck

DELAY_CREATE, DELAY_PROCESS, MAX_QUEUE, TIME = 1.0, 2.0, 5, 100_000

def run(channels, verbose_events=0):
    fun_rand.seed(1)
    create = Create("CREATE", DELAY_CREATE)
    process = Process("PROCESS", DELAY_PROCESS, channels=channels, max_queue=MAX_QUEUE)
    dispose = Dispose("DESPOSE")
    create.set_next(process)
    process.set_next(dispose)
    model = Model([create, process, dispose])
    model.simulate(TIME, verbose_events)
```

```

return model, create, process, dispose

def main():
    model, *_ = run(channels=3, verbose_events=5)
    model.print_result()

    print(f"\nВплив кількості пристроїв (T = {TIME}), імітація / теорія М/М/с/К:")
    print(f"{ 'с':>2} | { 'обслужено':>9} | { 'Рвідм':>15} | { 'завантаження':>15} | { 'черга':>15} | баланс")
    for channels in (1, 2, 3, 4, 5):
        _, create, p, dispose = run(channels)
        p_fail, mean_queue, mean_load = mmck(Delay_CREATE, Delay_PROCESS, channels,
        MAX_QUEUE)
        balance = (create.quantity == p.arrived == p.quantity + p.failure + p.queue +
        p.busy
        and dispose.quantity == p.quantity)
        print(f"{channels:>2} | {p.quantity:>9} | "
        f"{p.failure_probability():>7.4f}/{p_fail:<7.4f} | "
        f"{p.mean_load(TIME):>7.4f}/{mean_load:<7.4f} | "
        f"{p.mean_queue(TIME):>7.4f}/{mean_queue:<7.4f} | "
        f"{'так' if balance else 'HI'}")

if __name__ == "__main__":
    main()

```

Результат роботи

```

Подія в CREATE, t = 0.0000
CREATE: quantity = 1, tnext = 0.1443
PROCESS: busy = 1/3, queue = 0, quantity = 0, failure = 0, tnext = 2.8859
DESPOSE: quantity = 0, tnext = inf

Подія в CREATE, t = 0.1443
CREATE: quantity = 2, tnext = 0.4388
PROCESS: busy = 2/3, queue = 0, quantity = 0, failure = 0, tnext = 1.3381
DESPOSE: quantity = 0, tnext = inf

Подія в CREATE, t = 0.4388
CREATE: quantity = 3, tnext = 1.4931
PROCESS: busy = 3/3, queue = 0, quantity = 0, failure = 0, tnext = 0.6359
DESPOSE: quantity = 0, tnext = inf

Подія в PROCESS, t = 0.6359
CREATE: quantity = 3, tnext = 1.4931
PROCESS: busy = 2/3, queue = 0, quantity = 1, failure = 0, tnext = 1.3381
DESPOSE: quantity = 1, tnext = inf

Подія в PROCESS, t = 1.3381
CREATE: quantity = 3, tnext = 1.4931
PROCESS: busy = 1/3, queue = 0, quantity = 2, failure = 0, tnext = 2.8859
DESPOSE: quantity = 2, tnext = inf

----- РЕЗУЛЬТАТИ (T = 100000) -----
CREATE: quantity = 99969
PROCESS: arrived = 99969, quantity = 97968, failure = 2000
        mean queue = 0.5934, mean load = 0.6536, failure probability = 0.0200
DESPOSE: quantity = 97968

```

```

Вплив кількості пристроїв (T = 100000), імітація / теорія М/М/с/К:
с | обслужено | Рвідм | завантаження | черга | баланс
1 | 49612 | 0.5030/0.5039 | 0.9923/0.9921 | 4.0555/4.0630 | так
2 | 86531 | 0.1334/0.1333 | 0.8643/0.8667 | 2.0008/2.0000 | так
3 | 97968 | 0.0200/0.0203 | 0.6536/0.6531 | 0.5934/0.6002 | так
4 | 99565 | 0.0025/0.0027 | 0.5002/0.4986 | 0.1524/0.1553 | так

```

Таблиця 6 – Вплив кількості пристроїв на характеристики системи ($T = 100000$)

Пристроїв	Обслужено	Рвідм, імітація	Рвідм, теорія	R, імітація	R, теорія	L, імітація	L, теорія
1	49612	0,5030	0,5039	0,9923	0,9921	4,0555	4,0630
2	86531	0,1334	0,1333	0,8643	0,8667	2,0008	2,0000
3	97968	0,0200	0,0203	0,6536	0,6531	0,5934	0,6002
4	99565	0,0025	0,0027	0,5002	0,4986	0,1524	0,1553
5	99556	0,0003	0,0004	0,3969	0,3999	0,0365	0,0382

Висновок

З протоколу видно, що три вимоги, які надійшли одна за одною, обслуговуються одночасно трьома пристроями ($\text{busy} = 3/3$), а момент найближчої події елемента завжди дорівнює найменшому з моментів звільнення пристроїв. Зі збільшенням кількості пристроїв від 1 до 5 ймовірність відмови зменшується з 0,5030 до 0,0003, середня довжина черги — з 4,06 до 0,04, а завантаження одного пристрою — з 0,99 до 0,40.

Результати імітації збігаються з аналітичними значеннями для системи М/М/с/К при всіх значеннях s , баланс вимог виконується. Отже, модифікований клас Process коректно моделює багатоканальне обслуговування.

Завдання 6

Замість одного посилання на наступний елемент кожний елемент зберігає список маршрутів `routes` — пар «елемент, ймовірність переходу». Маршрути задаються методом `add_route()`; метод `set_next()` залишено для випадку єдиного наступного елемента. Наступний елемент вибирається методом `next_element()`: генерується рівномірно розподілене число ζ і вибирається той маршрут, у накопичений інтервал ймовірностей якого воно потрапило. Оскільки маршрут — це посилання на об'єкт, наступним може бути будь-який елемент моделі, зокрема попередній, тому повернення вимог не потребує окремої логіки.

Список маршрутів розміщено в базовому класі `Element`, тому розгалуження доступне і для `Process`, і для `Create`. Для демонстрації побудовано модель, схему якої наведено на рисунку 4: після `PROCESS 1` вимога з ймовірністю 0,6 переходить у `PROCESS 2` і з ймовірністю 0,4 — у двоканальний `PROCESS 3`; після `PROCESS 3` вимога з ймовірністю 0,7 залишає модель, а з ймовірністю 0,3 повертається в `PROCESS 1`.

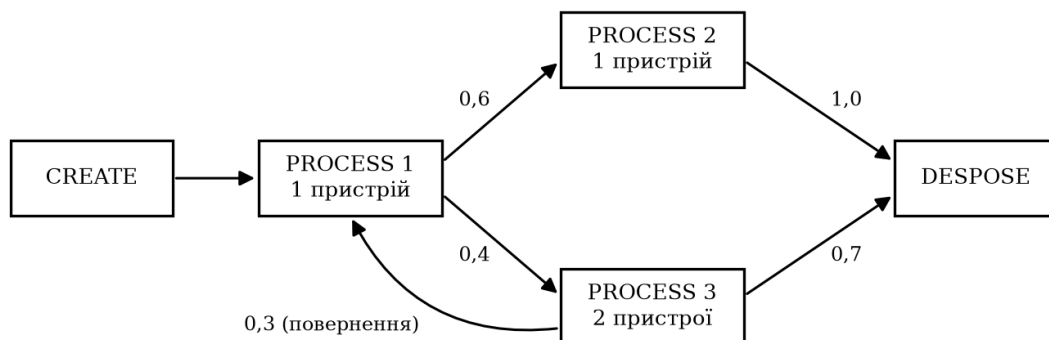


Рисунок 4 – Схема моделі з розгалуженням і поверненням вимог

Черги в цій моделі необмежені, тому її можна перевірити аналітично як мережу Джексона. Інтенсивності потоків у вузлах знаходяться з рівнянь балансу: $e_1 = \lambda + 0,3 \cdot e_3$, $e_2 = 0,6 \cdot e_1$, $e_3 = 0,4 \cdot e_1$, звідки $e_1 = \lambda / (1 - 0,4 \cdot 0,3) = 0,5682$ при $\lambda = 0,5$. Кожний вузол такої мережі поводить як незалежна система $M/M/c$ з інтенсивністю надходження e_i .

Лістинг файлів

Файл task6_routing.py:

```
"""Завдання 6: вихід у кілька наступних блоків і повернення у попередній блок.

CREATE -> PROCESS 1 -> PROCESS 2 (0.6) -> DESPOSE
        -> PROCESS 3 (0.4) -> DESPOSE (0.7)
        -> PROCESS 1 (0.3, повернення)

"""
import fun_rand
from create import Create
from dispose import Dispose
from model import Model
from process import Process
from theory import mmck

DELAY_CREATE, TIME = 2.0, 100_000
P_TO_2, P_TO_3, P_RETURN = 0.6, 0.4, 0.3

def run(verbose_events=0):
    fun_rand.seed(1)
    create = Create("CREATE", DELAY_CREATE)
    p1 = Process("PROCESS 1", 0.8)
    p2 = Process("PROCESS 2", 1.5)
    p3 = Process("PROCESS 3", 2.0, channels=2)
    dispose = Dispose("DESPOSE")

    create.set_next(p1)
    p1.add_route(p2, P_TO_2)
    p1.add_route(p3, P_TO_3)
    p2.set_next(dispose)
    p3.add_route(dispose, 1 - P_RETURN)
    p3.add_route(p1, P_RETURN)

    model = Model([create, p1, p2, p3, dispose])
    model.simulate(TIME, verbose_events)
    return model, create, (p1, p2, p3), dispose

def arrival_rates():
    """Рівняння балансу потоків:  $e1 = \lambda + 0.3 \cdot e3$ ,  $e2 = 0.6 \cdot e1$ ,  $e3 = 0.4 \cdot e1$ ."""
    e1 = (1 / DELAY_CREATE) / (1 - P_TO_3 * P_RETURN)
    return e1, P_TO_2 * e1, P_TO_3 * e1

def main():
    model, create, processes, dispose = run(verbose_events=6)
    p1, p2, p3 = processes
    model.print_result()

    returned = p1.arrived - create.quantity
    print("\nЧастоти переходів (імітація / задана ймовірність):")
    print(f"PROCESS 1 -> PROCESS 2: {p2.arrived / p1.quantity:.4f} / {P_TO_2}")
    print(f"PROCESS 1 -> PROCESS 3: {p3.arrived / p1.quantity:.4f} / {P_TO_3}")
    print(f"PROCESS 3 -> PROCESS 1: {returned / p3.quantity:.4f} / {P_RETURN}")

    print("\nПорівняння з теорією (мережа Джексона), імітація / теорія:")
    print(f"{'елемент':<10} | {'надійшло':>15} | {'завантаження':>15} | {'черга':>15}")
    for p, rate in zip(processes, arrival_rates()):
        _, mean_queue, mean_load = mmck(1 / rate, p.delay_mean, p.channels)
```

```

        print(f"{p.name:<10} | {p.arrived:>7}/{rate * TIME:<7.0f} | "
              f"{p.mean_load(TIME):>7.4f}/{mean_load:<7.4f} | "
              f"{p.mean_queue(TIME):>7.4f}/{mean_queue:<7.4f}")

    in_system = sum(p.queue + p.busy for p in processes)
    print(f"\nБаланс: створено {create.quantity} = вийшло {dispose.quantity} + у системі {in_system}: "
          f"'Так' if create.quantity == dispose.quantity + in_system else 'Ні'")

if __name__ == "__main__":
    main()

```

Результат роботи

```

Подія в CREATE, t = 0.0000
CREATE: quantity = 1, tnext = 0.2886
PROCESS 1: busy = 1/1, queue = 0, quantity = 0, failure = 0, tnext = 1.1544
PROCESS 2: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
PROCESS 3: busy = 0/2, queue = 0, quantity = 0, failure = 0, tnext = inf
DESPOSE: quantity = 0, tnext = inf

Подія в CREATE, t = 0.2886
CREATE: quantity = 2, tnext = 0.8775
PROCESS 1: busy = 1/1, queue = 1, quantity = 0, failure = 0, tnext = 1.1544
PROCESS 2: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
PROCESS 3: busy = 0/2, queue = 0, quantity = 0, failure = 0, tnext = inf
DESPOSE: quantity = 0, tnext = inf

Подія в CREATE, t = 0.8775
CREATE: quantity = 3, tnext = 2.0713
PROCESS 1: busy = 1/1, queue = 2, quantity = 0, failure = 0, tnext = 1.1544
PROCESS 2: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
PROCESS 3: busy = 0/2, queue = 0, quantity = 0, failure = 0, tnext = inf
DESPOSE: quantity = 0, tnext = inf

Подія в PROCESS 1, t = 1.1544
CREATE: quantity = 3, tnext = 2.0713
PROCESS 1: busy = 1/1, queue = 1, quantity = 1, failure = 0, tnext = 2.3980
PROCESS 2: busy = 1/1, queue = 0, quantity = 0, failure = 0, tnext = 1.1975
PROCESS 3: busy = 0/2, queue = 0, quantity = 0, failure = 0, tnext = inf
DESPOSE: quantity = 0, tnext = inf

Подія в PROCESS 2, t = 1.1975
CREATE: quantity = 3, tnext = 2.0713
PROCESS 1: busy = 1/1, queue = 1, quantity = 1, failure = 0, tnext = 2.3980
PROCESS 2: busy = 0/1, queue = 0, quantity = 1, failure = 0, tnext = inf
PROCESS 3: busy = 0/2, queue = 0, quantity = 0, failure = 0, tnext = inf
DESPOSE: quantity = 1, tnext = inf

Подія в CREATE, t = 2.0713
CREATE: quantity = 4, tnext = 3.2053
PROCESS 1: busy = 1/1, queue = 2, quantity = 1, failure = 0, tnext = 2.3980
PROCESS 2: busy = 0/1, queue = 0, quantity = 1, failure = 0, tnext = inf
PROCESS 3: busy = 0/2, queue = 0, quantity = 0, failure = 0, tnext = inf
DESPOSE: quantity = 1, tnext = inf

----- РЕЗУЛЬТАТИ (T = 100000) -----
CREATE: quantity = 50265
PROCESS 1: arrived = 57175, quantity = 57175, failure = 0
         mean queue = 0.3815, mean load = 0.4594, failure probability = 0.0000
PROCESS 2: arrived = 34249, quantity = 34246, failure = 0
         mean queue = 0.5315, mean load = 0.5124, failure probability = 0.0000
PROCESS 3: arrived = 22926, quantity = 22925, failure = 0
         mean queue = 0.0245, mean load = 0.2285, failure probability = 0.0000
DESPOSE: quantity = 50261

Частоти переходів (імітація / задана ймовірність):
PROCESS 1 -> PROCESS 2: 0.5990 / 0.6

```

PROCESS 1 -> PROCESS 3: 0.4010 / 0.4
PROCESS 3 -> PROCESS 1: 0.3014 / 0.3

Порівняння з теорією (мережа Джексона), імітація / теорія:

елемент	надійшло	завантаження	черга
PROCESS 1	57175/56818	0.4594/0.4545	0.3815/0.3788
PROCESS 2	34249/34091	0.5124/0.5114	0.5315/0.5351
PROCESS 3	22926/22727	0.2285/0.2273	0.0245/0.0248

Баланс: створено 50265 = вийшло 50261 + у системі 4: так

Таблиця 7 – Частоти переходів між елементами

Перехід	Частота в імітації	Задана ймовірність
PROCESS 1 → PROCESS 2	0,5990	0,6
PROCESS 1 → PROCESS 3	0,4010	0,4
PROCESS 3 → PROCESS 1 (повернення)	0,3014	0,3

Таблиця 8 – Порівняння імітації з теорією мережі Джексона (T = 100000)

Елемент	Надійшло, імітація	Надійшло, теорія	R, імітація	R, теорія	L, імітація	L, теорія
PROCESS 1	57175	56818	0,4594	0,4545	0,3815	0,3788
PROCESS 2	34249	34091	0,5124	0,5114	0,5315	0,5351
PROCESS 3	22926	22727	0,2285	0,2273	0,0245	0,0248

Висновок

Частоти переходів у імітації (0,5990, 0,4010 і 0,3014) збігаються із заданими ймовірностями 0,6, 0,4 і 0,3. Через повернення вимог у PROCESS 1 надійшло 57175 вимог при 50265 створених, що відповідає теоретичному коефіцієнту $1 / (1 - 0,12) = 1,136$. Баланс виконується: створено 50265 вимог, з моделі вийшло 50261, у системі на момент завершення залишилось 4.

Завантаження пристроїв і середні довжини черг у всіх трьох процесах збігаються зі значеннями, розрахованими за теорією мереж Джексона. Отже, механізм маршрутизації з розгалуженням і поверненням працює правильно, причому разом із багатоканальним обслуговуванням із завдання 5.

Висновок

У ході виконання комп'ютерного практикуму побудовано об'єктно-орієнтований алгоритм імітації мереж масового обслуговування, у якому модельний час просувається за принципом найближчої події, а стан моделі змінюється у моменти подій.

1. Реалізовано модель обслуговування одним пристроєм з обмеженою чергою та відмовами; додано обчислення середнього завантаження пристрою як відношення сумарного часу зайнятості до часу моделювання.
2. Побудовано модель із трьох послідовних процесів і виконано її верифікацію: перевірено баланс вимог, реакцію моделі на зміну кожного вхідного параметра та збіг характеристик першого процесу з аналітичними формулами M/M/1/K.
3. Клас Process модифіковано для обслуговування кількома ідентичними пристроями; результати збігаються з теорією M/M/c/K.
4. Реалізовано ймовірнісну маршрутизацію з виходом у кілька наступних блоків і поверненням у попередні; результати збігаються з теорією мереж Джексона.

Завдяки універсальному класу Element модель будь-якої структури збирається з однакових блоків без зміни алгоритму імітації: достатньо створити елементи, задати їхні параметри і з'єднати маршрутами. Порівняння з аналітичними моделями показало, що при часі моделювання 100000 розбіжності між імітацією і теорією лежать у межах статистичної похибки.

Посилання

1. Комп'ютерний практикум 2 «Об'єктно-орієнтований підхід до побудови імітаційних моделей дискретно-подійних систем»: методичні вказівки з дисципліни «Комп'ютерне моделювання дискретно-подійних систем». – КПІ ім. Ігоря Сікорського, 2026.
2. Law A. M., Kelton W. D. Simulation Modeling and Analysis. – 3rd ed. – McGraw-Hill, 2000.
3. Повний код практичної роботи: <https://github.com/ip34is/7-sem/tree/main/CMDES/CMDES-lab2>